UNIVERSIDADE DE AVEIRO

DEPARTAMENTO DE ELECTRÓNICA, TELECOMUNICAÇÕES E INFORMÁTICA

Algorithmic Information Theory (2025/26)

Project #2

To deliver: 26 May 2026 (18:00)

Introduction

Lossless data compression methods can broadly be divided into two major classes: *general-purpose* compressors and *data-specific* compressors. General-purpose compressors are designed to perform reasonably well across a wide range of inputs, without assuming strong prior knowledge about the source. Typical examples include tools such as `gzip`, `bzip2`, `xz`, or `zstd`, which aim at robustness, portability, and good average performance over heterogeneous data.

In contrast, data-specific compressors are designed for a narrower class of sources and exploit structural properties that are expected to occur in that domain. When the nature of the data is known in advance, such specialization can provide significant gains in compression ratio, computational efficiency, or both. These gains arise from the ability to incorporate stronger assumptions about symbol distributions, spatial or temporal dependencies, acquisition artifacts, noise characteristics, or other regularities that would not be accessible to a purely generic compressor.

This distinction is especially relevant in scientific data compression. Raw data acquired from a specific sensing process often exhibits highly characteristic statistical patterns that differ substantially from those of everyday files such as text, executables, or multimedia containers. As a result, a compressor tailored to a known data modality may substantially outperform a general-purpose alternative, provided that the model captures the relevant structure while remaining computationally practical.

In this project, students are invited to explore this trade-off between generality and specialization in the context of raw astronomical image data. The challenge is not only to build a correct lossless compressor/decompressor pair, but also to design a method that takes advantage of the expected properties of the target data class and to assess, through benchmarking, the extent to which such specialization improves over standard general-purpose compression tools.

The challenge

Within the framework of Algorithmic Information Theory, Project #2 consists of designing and implementing a *lossless* data compression tool for raw spatial images provided by the teachers. The deliverable must include both a compressor and a corresponding decompressor.

The input data consists of image samples extracted from astronomical observations obtained with several ground-based large telescopes. Each sample is stored as a raw raster with:

- spatial resolution of 1500×1500 pixels;
- 16 bits per pixel;
- unsigned integer representation;
- big-endian byte order.

The modeling and coding components are intentionally left open-ended, provided that all project requirements are satisfied. This flexibility allows each group to explore different trade-offs between compression ratio, computational cost, memory usage, and implementation complexity. Possible approaches may include predictive models, context-based schemes, dictionary methods, invertible transforms, statistical encoders, or hybrid solutions. The design of both the modeling and coding stages is left open, as long as the final system is fully lossless and satisfies all project requirements.

Although not mandatory, an implementation in an efficient systems programming language (e.g., C, C++, or Rust) is strongly recommended to ensure good performance on realistic inputs.

A `.zip` archive containing several files will be provided as a benchmark suite for development and preliminary evaluation (training data). After submission, the compressors will also be evaluated on a separate testing dataset with similar characteristics.

An updated online benchmark (refreshed during or after each class) is available at:

`https://pratas.github.io/rai-challenge.html`

Requirements

1. Students will work in groups of three students.
2. The compressor must be *lossless*: only solutions that reconstruct the original data exactly after decompression will be considered.
3. The implementation must run in Linux, using a maximum of 8 GB of RAM.
4. The compression binary must not exceed 5 MB.
5. The decompression binary must not exceed 1 MB.

6. A benchmark against state-of-the-art compressors (e.g., `gzip`, `bzip2`, `lzma/xz`, `zstd`) must be provided, reporting at least:
   - compressed size (bytes),
   - compression rate per symbol (e.g., bits/symbol or bits/byte),
   - compression time,
   - decompression time.
7. The evaluation will emphasize your ability to synthesize concrete ideas, explanations, comparisons, and results. You must submit a report of at most 6 pages in PDF format, using the following template: template.
8. Each group must give a poster presentation in a conference-style format (no need to print the poster, only to have the PDF), using at most 6 minutes. All group members must participate in the presentation and discussion of the work.
9. The compression/decompression tool, the respective code, and the report must be submitted via the e-learning platform.

Evaluation

- We will evaluate your feedback/progress during the classes. The idea is for groups to use the practical class time not only to develop the project, but also to discuss relevant aspects with the teachers.
- Extra credit will be given to new methodologies/implementations and to high flexibility of the compressor with respect to the input data.
- Compression/decompression performance will be evaluated using a custom metric, which will only be disclosed after submission. This metric is intended to balance compression capability, compression speed, and decompression speed.
- We will also evaluate the overall quality of the solution, including readiness to use, robustness, and documentation.
- Finally, we will evaluate your ability to synthesize the main ideas, methodologies, comparisons, and results in the written report and poster presentation.

Data

In Moodle, a zipped file is available in the `Project #02` folder:

- `data2.zip` — benchmark data for development and evaluation.

The training data is composed of samples of astronomical images acquired by several ground-based telescopes. All files correspond to raw image patches of size 1500×1500 pixels, represented as unsigned 16-bit integers in big-endian order. These samples were extracted from larger images and preserve the statistical and structural properties of the original observations.

The training set contains 8 files extracted from astronomical observations acquired by different ground-based telescopes, including VLT, TJO, GTC, INT, JKT, LCO, and WHT. These files were selected to provide a diverse benchmark with different image characteristics, noise levels, and spatial structures.

Students should take into account that these files are not conventional image formats (such as PNG, JPEG, or TIFF), but raw numerical arrays. Therefore, the compressor must correctly interpret the input as a two-dimensional field of 16-bit samples, while preserving exact reversibility.

FAQ

- Can we use AI-based tools? Yes, but we recommend moderate and responsible use. Keep in mind that *novelty* will be strongly valued throughout the project and during the classes. Moreover, you will be assessed on your ability to clearly explain the implemented approach and the relevant parts of the code.

- Can we exceed the intended space for the poster or 6 pages for the report (e.g., to include more detail)? This is *strongly discouraged*. Respecting space constraints is part of the evaluation, namely your ability to synthesize and communicate the key ideas efficiently.

- Can we use code from other authors? Yes, provided it is properly referenced (author credit is essential). This may be relevant for low-level components of the compressor, libraries, or auxiliary modules. However, the core ideas, system integration, and the main modeling/compression strategy should be developed and clearly understood by the group.